

Cracking the Code: Can Combinatorics in Mathematics be used to Determine an Ideal Method of Cracking Passwords in Mastermind?

#13

February 11, 2024

Introduction:

There aren't many people that would find a 50 year old puzzle game to be intriguing, much less one that never gained topmost popularity, but one game was able to catch my eye. Mastermind was invented in the 1970s by an Israeli telecommunications expert named Mordecai Meirowitz. It is said that he was inspired by MOO, a project developed by J. M. Grochow at MIT in the late 1960s¹, and based his game on another two player paper game called Bulls and Cows.

Mastermind is a two player puzzle game played using colored pegs. One player, assigned as the code maker, creates a hidden "code" using any four out of eight different colored pegs. No color may be used more than once in the code. The objective is for the other player, the codebreaker, to guess the code within twelve turns. There are no restrictions on the guesses the codebreaker can make. The code maker will give feedback for every guess, telling the codebreaker the number of colors they guessed correctly, and the number of pegs both of the right color and placed into the right position. What the code maker will not tell you is which specific pegs they are. Although there are multiple versions of this game, for the sake of this investigation, the rules mentioned above will be the version I center my exploration around.

Mastermind hasn't only been a game I enjoyed, it's a long lasting childhood memory. I first came across it as a kid, and would beg my family to play it with me. I had always felt a strange pull towards puzzle games and I played many different versions in the years to follow. However, I never fully understood how complicated the game really was. So when tasked to complete an exploration in class, I immediately thought to use this chance to explore this interest of mine. It would be fun to apply math I'd learned in class to a real life game and finally be able to understand everything behind it.

¹ Fyfe, Duncan. (2020). Mastermind, *The Codebreaking Board Game*. Retrieved from Vice January 2024.

Statement of the Task:

The codebreaker game is a fun puzzle game for all ages as you can increase the difficulty by changing the rules. Leslie Ault, author of *The Official Mastermind Handbook*, states that the success of Mastermind is straightforward because “It’s easy to learn, plays quickly [and] requires no translation and rewards.”² Though the aim of the game is simple, it includes many mathematical patterns, which will be explored and analyzed.

Combinatorics is the area of mathematics concerned with counting strategies and arranging objects.³ This will be the central idea of my exploration. I will specifically focus on cryptanalysis, which is the process of finding weaknesses in codes and using these to decipher text without knowing the “secret code”, which is the concept of codebreaker.⁴ My goal is to thoroughly analyze the processes involved in cracking the code in the codebreaker puzzle game; including, but not limited to, using permutations and combinations for calculating the efficiency of guesses, finding guess distribution and likeliness of solving a code in the specified amount of turns, as well as looking into online passwords. Through this, I will come to a conclusion of whether or not the strategy I will analyze is effective in solving the code every single time.

Introduction to the Investigation:

In my experience with this puzzle game, one approach is most successful in cracking the code within the maximum of twelve attempts. This approach splits all arrangements into two sets: Set A, where the codebreaker has all the right colors for the code but are in the wrong position, and Set B, where the colors of the code are not yet determined. To best illustrate these mathematical processes, I will center my exploration on solving the code from Set A, although I will briefly touch on reaching Set A from Set B.

² Fyfe, Duncan. (2020). Mastermind, *The Codebreaking Board Game*. Retrieved from Vice January 2024.

³ Earl, R., & Nicholson, J. (2021). *The Concise Oxford Dictionary of Mathematics*. Oxford University Press. Pg.67

⁴ Earl, R., & Nicholson, J. (2021). *The Concise Oxford Dictionary of Mathematics*. Oxford University Press. Pg.92

Permutations and Combinations in Mastermind:

Mastermind uses many mathematical concepts, primarily permutations and combinations.

Permutations are an arrangement of terms where the order matters (Solving the code from Set A) and Combinations are a grouping where the order of elements is unimportant (Reaching Set A from Set B).

Here, the order of the colors matters. This is a permutation. The number of total possible codes in this version of the game is calculated using the permutation formula as follows. “n” is the number of total colors to choose from and “r” is the number of colors to be selected from the total. n^P_r is the number of ways to arrange any “r” digits out of “n”.

$$n^P_r = \frac{n!}{(n-r)!}$$
$$8^P_4 = \frac{8!}{(8-4)!} = \frac{8!}{4!} = 1680$$

There are 1680 possible codes made from arranging any four colors out of eight. This means if you were to randomly guess, you would have a 12 in 1680 chance of getting the code correct in the twelve allowed turns. That's only around 0.71%, clearly this is an inefficient method.

Since my investigation will focus on what happens after guessing the right colors. This substitution will be used instead, where you permute four already known terms.

$$4^P_4 = \frac{4!}{(4-4)!} = 4! = 24$$

There are a total of 24 ways to arrange the four correct colors established prior, given that we already know the right colors.

Similar to the two calculations above, the total number of combinations is represented below, where you choose 4 colors out of 8 (order doesn't matter). Again, "n" is the number of total colors to choose from and "r" is the number of colors to be selected from the total.

$${}_nC_r = \frac{n!}{(n-r)! r!}$$

$${}_8C_4 = \frac{8!}{(8-4)! 4!} = \frac{8!}{4! 4!} = 70$$

Instead of going straight to finding 1 out of 1680 Permutations, I will prioritize finding the right combination out of 70 before finding 1 out of the 24 permutations of that combination.

For further analysis, I will introduce the format of feedback given by the code maker that I will be using throughout the rest of my exploration. I will use the letters C (for correct color), and B (for both correct color and correct position). For example: "0C4B" will mean the code is solved and "4C0B" means you have the right combinations of colors all in the wrong spots. I then realized that the feedback "1C3B" is impossible. One cannot have four colors, three of which are in the right position because there is only one space left open after you place the three.

Reaching Set B:

You want to reach Set B in the least moves possible because there are still multiple possibilities to rule out. Since no repeats are allowed in this version, the worst case scenario is 8 guesses because you can guess one color each turn by guessing four pegs of the exact same color. By the end of the eight you will have reached Set A no matter what. There are multiple better ways to do this such as guessing more colors at a time and analyzing all feedback, however the worst case scenario is most straightforward and sufficient because solving the code after finding the colors only takes 4 guesses, (perfectly adding up to the limit of 12 turns) as I will demonstrate later on.

Determining Efficiency of Different Feedback:

There are 24 ways to arrange the colors established. I have represented the four colors with the letters W, X, Y and Z. The different arrangements are shown below in **Figure 1**.

WXYZ	WXZY	WYZX	WYXZ	WZYX	WZXY
XYZW	XYWZ	XZWY	XZYW	XWYZ	XWZY
YWXZ	YWZX	YZWX	YZXW	YXZW	YXWZ
ZWXY	ZWYX	ZYWX	ZYXW	ZXWY	ZXYW

Figure. 1

The four results the codebreaker could get: 2C2B, 3C1B, 0C4B and 4C0B. 0C4B is the goal of the game, and is irrelevant to consider because you have already reached the goal.

I will use the first case as an example to screen out possible permutations. Say the codebreaker guessed the code WXYZ and got the feedback 2C2B. All possible permutations that cannot be made by exchanging the positions of two colors can thus be eliminated as shown below. The original code, WXYZ, is eliminated as well.

WXYZ	WXZY	WYZX	WYXZ	WZYX	WZXY
XYZW	XYWZ	XZWY	XZYW	XWYZ	XWZY
YWXZ	YWZX	YZWX	YZXW	YXZW	YXWZ
ZWXY	ZWYX	ZYWX	ZYXW	ZXWY	ZXYW

Figure. 2

This process eliminates eighteen permutations leaving six. The screening efficiency is a measure of how effective the screening process is and is calculated by taking the number of perms eliminated over the twenty-four possible. The screening efficiency for this is 75%.

I played multiple sample rounds of Mastermind to come up with a pattern which helped me get to know the whole game a lot better from a mathematical standpoint. The same screening strategy can be used with other feedback. If you get 4C0B then you can eliminate all permutations that have any same letter in the same position as the code you have guessed. When you get the result 3C1B, you can eliminate all the derangements of your guess, you can also eliminate the ones that have exactly two terms in common with your guess. This leaves you with eight permutations.

The screening efficiency for the three cases we must consider are listed in **Figure 3**.

Screening Feedback and Efficiency of each when screening out codes.

Screening Feedback	Screening Efficiency
2C2B	$18/24 = 75\%$
4C0B	$15/24 = 62.5\%$
3C1B	$16/24 \approx 66.7\%$

Figure.3

Derangements:

Derangements, as an extension of permutations, are rearrangements of a permutation such that none of the terms in the original permutation go back to their original position.⁵ The concept of derangements is most efficient to find the correct order of colors when given the feedback 4C0B.

Multiple sources I found denoted derangements as $!n$, whereas others used the symbol D_n .

In my calculations, I will be using D_n because it is unlikely to be confused with factorials whereas $!n$ might⁶.

⁵ Earl, R., & Nicholson, J. (2021). *The Concise Oxford Dictionary of Mathematics*. Oxford University Press. Pg.104 "Derangement"

⁶ *Derangements*. (n.d.). *Art of Problem Solving*.

The first conjecture:

I wanted to try and create my own conjecture for the number of derangements so I started with simple derangements to find a pattern. I noted that there is one derangement of a two digit code and two derangements of a three digit one. From this, I concluded that derangements are affected by the number of terms. The conjecture I have made is simply

$$D_n = n - 1$$

Where D_n is the number of derangements of a set with n terms.

I thought this was accurate until I listed out all the derangements of four terms and found that there were actually 9 instead of 3.

The second conjecture:

I noticed that the number of derangements must be less than the number of total arrangements. The number of derangements should theoretically be:

number of total arrangements – number of arrangements with one term in the same spot

This is my second conjecture:

$$D_n = {}_n P_n - {}_n C_1 (n-1)!$$

First, choose one term to be fixed in its original position(${}_n C_1$). Then, there are $(n-1)!$ Ways to arrange the other terms. Again, I was not able to obtain the correct answers using this formula meaning there must still be something wrong with it. I realized a bigger flaw in this conjecture because ${}_n P_n = {}_n C_1 (n-1)!$ so when you subtract those, the result is always zero.

The third conjecture:

I made sure to do research before coming up with a third conjecture. The maximum number of permutations must be $n!$. Then, you subtract permutations that don't apply from there. After subtracting the ones with one element fixed, you still must add back the permutations with two elements in their original positions. This is because you might count a permutation twice or more which means you have to subtract the ones with three in their original spots and so on. This creates an alternating pattern of addition and subtraction. It can be said that all odd terms in the conjecture are added and every even term is subtracted; ending with the term with $n C_n$.⁷⁸

$$\begin{aligned} D_n &= n! - {}_n C_1 (n-1)! + {}_n C_2 (n-2)! \dots (-1)^n {}_n C_n (n-n)! \\ &= n! \left(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \dots + (-1)^n \frac{1}{n!} \right) \\ &= n! \sum_{k=0}^n \frac{(-1)^k}{k!} \end{aligned}$$

In this version of the codebreaker game, $n = 4$ because the code is made up of four digits.

$$\begin{aligned} D_4 &= 4! \left(1 - 1 + \frac{1}{2} - \frac{1}{6} + \frac{1}{24} \right) \\ D_4 &= 9 \end{aligned}$$

Substituting $n = 4$ into the equation results in a value of 9. With the code being made out of 4 digits, and assuming you know the colors in the code, if you guess a code and the feedback received is 4C0B, there are 9 other possibilities of the code that are wise to guess from.

⁷ M.T.L Bizley, A Note on Derangements

⁸ *Derangements*. (n.d.). *Art of Problem Solving*.

How many turns are needed to solve the code from Set A?

We only have to consider the least efficient case because all other cases may be solved in fewer turns than that one. This is the case where the screening efficiency is 62.5%.

The complementary probability formula states “ In any experiment, an event A or its complement must occur.”⁹ Since the screening efficiency is 0.625, we can subtract that from 1 to get 0.375. This new value represents the percentage of valid codes remaining after screening.

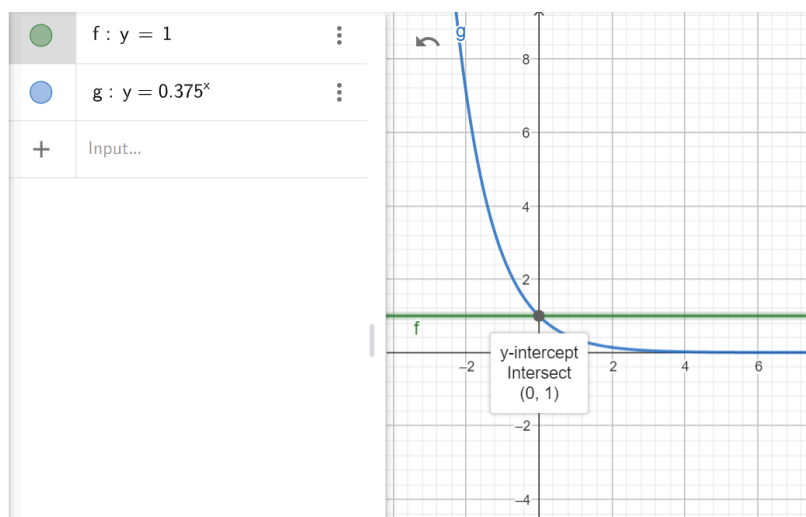
$$P(\text{getting correct code in } n \text{ guesses}) = (1 - \text{percentage})^n \\ = (0.375)^n$$

“n” is the number of trials which must be greater than or equal to 1 and the percentage is the least efficient screening efficiency. The exponent n is equivalent to multiplying this probability multiple times. This approximation considers the worst case scenario because every turn you will get one type of feedback each turn which creates a cumulative effect. To find the “n” value where probability is equal to 1, I created a system of equations to find the intersection of these two functions:

$$y_1 = 1$$

$$y_2 = (0.375)^x$$

In calculator format, the variable x is used. It is the same variable as I am representing with “n”.



⁹ Watts, Valerie. (2022). *Ecampus Ontario Pressbooks. The Complement Rule.*

I discovered that I was wrong again. The two functions intersect at $n=0$ which I should've realized since the x-intercept of an exponential graph without a vertical transformations is always at $(0, 1)$.

After some further thinking I multiplied everything with a vertical stretch of 24 about the x-axis. I thought about doing this because I realized the 0.375 being multiplied was out of a total of 1 guess which would not apply to our case. Essentially, you scale the result to make it out of twenty four guesses and not limited to one.

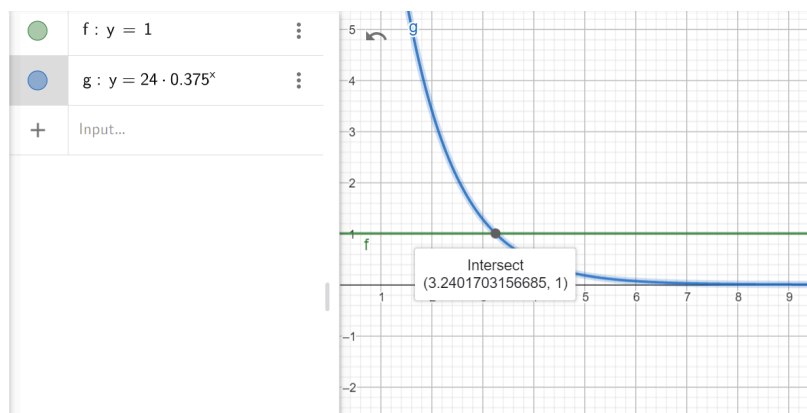
$$P(\text{getting the right code in } n \text{ guesses}) = 24(0.375)^n$$

I will solve this first using an algebraic approach and can later use a technological one to verify my results. Same as above, set the probability equal to 1.

$$1 = 24(0.375)^n$$

$$\log\left(\frac{1}{24}\right) = n\log(0.375)$$

$$n \approx 3.2402$$



By setting this new function equal to 1, the intersection results in an n value of approximately $n \approx 3.240$ which I would round up to the nearest greater whole. This is because in some cases you will need a little more than 3 guesses.

Therefore a maximum of four guesses are required to solve the code from this stage. Both approaches produce the same result, so my answer is verified.

One thing I noticed when re-reading my investigation is that my graphs above are decreasing. With more turns the probability is seemingly less when it should be more because getting the code in the first two turns is unlikely. However, I still believe this version to be accurate for the maximum number of turns because I have run multiple trials with family and I can always get the right code in four turns or less. Since this is a major limitation to this model, I will consider a different approach to model the average number of turns to solve the code from Set A.

What is the average number of guesses needed to solve the code from Set A?

I will be using another, non-graphical approach to find the average guesses needed to crack the code at this step. Since in my previous approach, I assumed the least efficient screening for every guess and didn't consider that prior feedback could alter the probability of the next value, this is a better model.

Assuming the codebreaker will get all the colors correct on the next turn, a probability analysis can be used to analyze the probability of cracking the code and to find the number of turns needed to crack the code from this turn.

$$P(\text{An event occurring}) = \frac{\# \text{ of favorable outcomes}}{\# \text{ of total outcomes}}$$

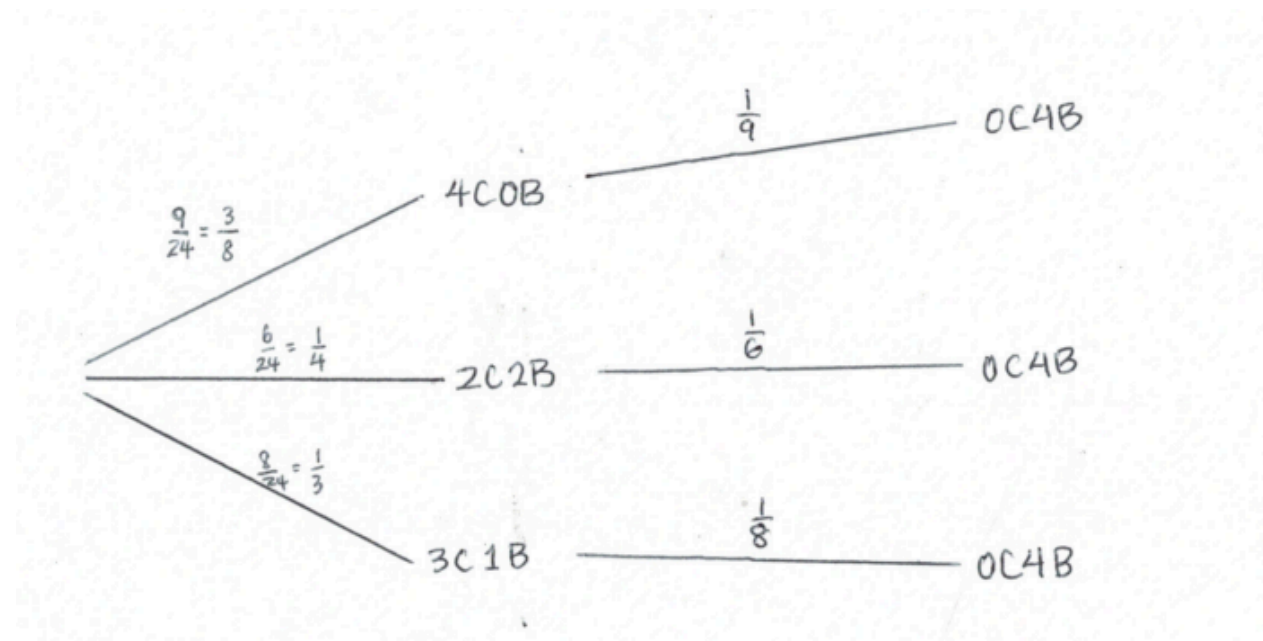
$$P(\text{correct code on first guess}) = \frac{1}{24} \approx 0.0417 \approx 4.17\%$$

There is a 4.17% chance that you will get the correct code on the first try; evidently a rare case.

I initially thought that since there is a probability of one in twenty four of getting it on the first try, there would be a probability of one in twenty three on the second. This is because you know that the first guess was incorrect, eliminating one out of twenty four total permutations.

Theoretically this would make sense, it is more likely to solve the code in two tries than in one because solving it in one would take a large stroke of luck. However, there was one essential component that I forgot to consider: which was that the code maker provides additional feedback after each guess. This would completely change the way probability is calculated. It is actually a little more complicated than that. An assumption I have made is that the feedback provided by the code maker is authentic. One piece of misleading information will throw this strategy off its course.

There is a greater probability of getting it on the second try because you have feedback obtained in the second turn. You would calculate the probability of getting each feedback and multiplying it with the probability of getting the correct code after knowing the feedback.



$$P(\text{correct code on second guess}) = \left(\frac{1}{9} \times \frac{9}{24} \right) + \left(\frac{1}{6} \times \frac{6}{24} \right) + \left(\frac{1}{8} \times \frac{8}{24} \right) = \frac{1}{8} = 12.50\%$$

The probability of getting the code on the third guess onwards is difficult to illustrate because there are so many cases to consider. From our previous calculations, we know that 4 is the maximum number of turns needed. Any code not solved in the first two turns will definitely be solved in the third or fourth turn. The probability of solving on these turns can be found by subtracting the probabilities of all the other turns from 1. This is another demonstration of complementary probability¹⁰. However this is also not optimal as the calculations also become an approximation from combining the 3rd guess with the 4th. To do this, I have taken the average between the two resulting in an X value of 3.5.

$$P(\text{correct code on third or fourth guess}) = 1 - P(\text{correct code on first guess}) - P(\text{correct code on second guess})$$

$$P(\text{correct code on third or fourth guess}) = 1 - \frac{1}{24} - \frac{1}{8} = \frac{5}{6} \approx 0.833$$

Now I will calculate the expected number of guesses needed to solve the code from Set A. This will be done by setting the random variable x to the number of guesses. I have not considered zero turns because the codebreaker has no prior feedback.

Probability Distribution table of the number of turns taken to crack the code.

X	1	2	3.5
$P(X = x_i)$	$\frac{1}{24}$	$\frac{1}{8}$	$\frac{5}{6}$

$$E(X) = \sum_{i=1}^N x_i \times P(X = x_i)$$

$$E(X) = 1\left(\frac{1}{24}\right) + 2\left(\frac{1}{8}\right) + 3.5\left(\frac{5}{6}\right)$$

$$\approx 3.208$$

¹⁰ Watts, Valerie. (2022). *Ecampus Ontario Pressbooks. The Complement Rule.*

The expected value of the random variable X rounded to the nearest whole is 3. This means the average amount of moves needed to solve the code from Set A is 3. In this case, I have rounded down because the average is closer to 3. We wouldn't round up as we did when we solved for the maximum turns because we do not have to consider the extreme cases.

A consideration should be made that there are multiple other strategies out there, which are much more efficient than mine. However, this strategy appealed to me when deciding because focusing on too many would quickly make the investigation confusing. This specific one could be modeled using mathematical processes that would make sense to high school students.

I have determined that using this specific strategy of mine, the code can certainly be solved within 12 attempts if the codebreaker thoroughly evaluates all feedback provided. Determining the colors taking up to 8 guesses at worst and finding the right order of the colors in a maximum of 4 moves. My aim was to see if one could always solve the code using this strategy and to analyze the processes within the game. This investigation has done just that! However, what really interested me was how the Louisville Courier Journal predicted that "It is reasonable to expect that people playing electronic mastermind will be mistaken for hard workers laboring over their calculators." This game really attracts people to it.

I have thoroughly analyzed multiple aspects of Mastermind, a seemingly simple board game with mysterious origins. This was in order to truly "hack" the game and to find out all the tricks, all the ins and outs of puzzle games involving combinatorics. My interest in Mastermind started many many years ago, but I've still never turned down a game request from my little sister to this day! When Wordle came out in 2021, I also had a great interest in it. Only after completing my exploration did I realize that Wordle was based on Mastermind!

The biggest thing I learnt was how mathematics are present in our daily lives. When exploring the mathematics of this game, I learned to see things in our daily lives from a mathematical standpoint and that all things, even ones we don't normally take note of, can be explained using

mathematics. I managed to extract so much information to analyze from a single, simple codebreaker game that I need not imagine how crucial math is in just about everything we do.

Conclusion:

Seeing how easily codes could be broken in a game, even a simple code, surprised me. All the times I sat there thinking hard about how to win Mastermind as I was running out of turns, but failing at the end, disappointed me. The purpose of this investigation was to investigate the mathematics behind codebreaker which helped me fully understand and realize that the code is certain to be solved within the allocated moves if only one stopped to really delve deep into the game as I have done.

With modern technology as it is today, a simple code could be broken instantaneously. This prompted me to consider real life scenarios in which cryptanalysis is involved. And this really shook me that my passwords were not at all safe, I was just lucky not to be hacked! Nowadays, programs are capable of testing anywhere from ten thousand to a billion codes per second.¹¹ The importance of good passwords is real! For example with a 4 digit code there are merely 1680 possible codes, which would be hacked instantaneously. Adding digits to your password does not essentially guarantee higher security because it's simple for hackers to guess any password instantaneously up to about 11 digits¹². I don't know many people who make their passwords that long! I'm online a lot and this information encouraged me to change my passwords too.

I have to say that I feel a great sense of accomplishment after finishing this investigation. All the time and effort put into the whole thing was definitely worth it. I hope to also look into some other interesting games such as Sudoku or Wordle. They are all games and puzzles I used to spend countless hours working on. Creating a puzzle of my own would be very fun!

¹¹ Neskey, Corey. (2023). *Are Your Passwords in the Green?*

¹² Neskey, Corey. (2023). *Are Your Passwords in the Green?*

References:

1. Earl, R., & Nicholson, J. (2021). *The Concise Oxford Dictionary of Mathematics*. Oxford University Press.
2. Bizley, M. T. L. (1967). *A Note on Derangements*. The Mathematical Association. Retrieved from Jstor (January 2024)
<https://www.jstor.org/stable/3614384?searchText=derangements&searchUri=%2Faction%2FdoBasicSearch%3FQuery%3Dderangements&ab_segments=0%2Fbasic_search_gsv2%2Fcontrol&refreqid=fastly-default%3Af18b1ef3bc17ca6e3439a427ef8a439d>
3. Weisstein, Eric W. *Mastermind*. From *MathWorld*--A Wolfram Web Resource.
<<https://mathworld.wolfram.com/Mastermind.html>>
4. Fyfe, Duncan. (2020). *Mastermind, The Codebreaking Board Game*. Retrieved from *Vice* January 2024.
<<https://www.vice.com/en/article/884k54/permalink-mastermind-board-game-50th-anniversary-origins-fallout-cybersecurity>>
5. *Derangements*. (n.d.). *Art of Problem Solving*. Available at:
<https://artofproblemsolving.com/wiki/index.php/Derangement> (Retrieved January 2024).
6. Watts, Valerie. (2022). *Ecampus Ontario Pressbooks. The Complement Rule*.
<<https://ecampusontario.pressbooks.pub/introstats/chapter/3-4-the-complement-rule/>>
7. *Calculator Suite*. GeoGebra. (n.d.). <<https://www.geogebra.org/calculator>>
8. Neskey, Corey. (2023). *Are Your Passwords in the Green?*
<<https://www.hivesystems.io/blog/are-your-passwords-in-the-green>> (Retrieved February 2024)